

ShopKart — Mini Project Build Prompts

Two prompts to hand to Claude Code, one per app. Build and run them in separate project folders, on separate local ports, so you can test the full flow between them.

Setup Note

Create two project folders: **ecommerce-app** and **admin-app**. Run the E-Commerce App on **port 3000** and the Admin App on **port 4000**. Both apps must read and write to the **same database** — this is what lets the Admin App look up a customer and see exactly what they bought, and look up a product and see exactly who bought it. Build the E-Commerce App first, since the Admin App depends on data created by it.

1. E-Commerce App Prompt

Copy everything in the box below and give it to Claude Code inside the **ecommerce-app** folder.

Build a customer-facing e-commerce web app called "ShopKart" in this folder, running on port 3000. It should include both the frontend and the backend needed to make it fully functional, backed by a database.

The app should let a customer do the following, start to finish:

Account

- Sign up with name, email, phone, and password, and log in / log out.
- View and edit their profile.
- Save multiple delivery addresses and mark one as default.

Browsing products

- View a home page with featured or latest products.
- Browse products by category.
- Search products by name or keyword.
- Filter products by category, price range, rating, and availability.
- Sort products by price, newest, or rating.
- View a product's detail page, including its variants (like size or color), price, description, images, and customer reviews.

Cart

- Add a product (with a chosen variant) to their cart, and adjust quantity or remove items.
- See a live cart summary: subtotal, any discount, tax, shipping fee, and total.
- Apply or remove a discount coupon code.

Checkout and orders

- Choose a delivery address and place an order from their cart.
- Simulate a payment step (a simple mock "pay now" that marks the order as paid — no real payment gateway needed).

- View their order history with each order's current status (for example: placed, paid, packed, shipped, delivered, cancelled).
- View a single order's details, including exactly which products and quantities were in it.
- Cancel an order if it hasn't shipped yet.
- Track a shipped order's delivery status.

Reviews

- Leave a star rating and written review for a product, but only if they have actually received (delivered) that product in a past order.
- Edit their own review for a short window after posting it.

Important design rule

Every purchase must be recorded in a way that clearly links: this specific user → this specific order → these specific products and quantities → at the price they paid at that time. This record should never be lost or overwritten even if the product's price or name changes later. This is the data the Admin App will depend on, so make sure it's structured cleanly and is easy to query by user or by product later.

Keep the design clean and simple — this is a personal practice project, so prioritize the flows working correctly over visual polish. Show clear, friendly error messages when something goes wrong (out of stock, invalid coupon, expired session, etc).

2. Admin App Prompt

Copy everything in the box below and give it to Claude Code inside the **admin-app** folder, after the E-Commerce App is built.

Build an internal admin dashboard called "ShopKart Admin" in this folder, running on port 4000. It connects to the same database as the E-Commerce App (in the ecommerce-app folder), so it can see the real customers, products, and orders created there.

Only allow login for admin-role users, not regular customers.

The admin should be able to do the following:

Product and inventory management

- View, add, edit, and archive products and their variants.
- Adjust stock levels for a specific variant/SKU.

Order tracking

- View all orders with their current status.
- Update an order's status as it moves through fulfillment (packed, shipped, delivered) and attach tracking info.

The core feature — user and product traceability

This is the main purpose of the admin app, so it should be easy to find and use:

- Search for a customer by name, email, phone, or ID, and pull up a full list of every product they have ever purchased — including quantity, price paid, order number, order status, and purchase date.
- Search for a product and pull up a full list of every customer who has ever bought it — including quantity, order number, and purchase date.
- A combined search screen where the admin can filter purchase records by customer, product, SKU, order status, and date range, and export the results (for example, to CSV).

Reporting

- A simple reports view showing top-selling products, revenue by day, and repeat-customer rate.

Important design rule

Never link a user to a product directly. Always trace the connection through their actual orders, so the history stays accurate even if the product details change later. Make the two traceability screens (user → their purchases, and product → its buyers) feel like two sides of the same coin, since proving that link both ways is the whole point of this app.

Keep the design clean, table-heavy, and functional — this is an internal tool, not a customer storefront, so prioritize clear data over visual polish.

3. How to Test It End-to-End

- 1 In the E-Commerce App: sign up as a customer, browse products, add one to the cart, checkout, and pay.
- 2 In the Admin App: log in as admin, search for that customer, and confirm the product you just bought shows up under their purchases.
- 3 Still in the Admin App: search for that same product and confirm the customer shows up in its buyers list.
- 4 Cancel the order in the E-Commerce App, then confirm it's reflected as cancelled in the Admin App.